Complemento Práctico: Dando Vida a los Objetos

En el apunte teórico definimos que un objeto tiene "Estado, Comportamiento e Identidad". Pero, ¿cómo se ve esto en el código real? En esta guía práctica, analizaremos el poder del **operador punto (.)**, descubriremos qué valores tienen los atributos antes de que nosotros los toquemos (Valores por Defecto) y aprenderemos a escribir métodos inteligentes que no solo imprimen mensajes, sino que **toman decisiones** basándose en el estado actual del objeto.

Contenido

El operador punto como mecanismo de acceso

Cómo interactuar con atributos y métodos de un objeto mediante el operador punto

Inicialización automática de atributos

Los valores por defecto que Java asigna según el tipo de dato

Métodos que modifican el estado interno

El verdadero poder de los métodos más allá de imprimir mensajes

Validación de precondiciones en métodos

Programación defensiva: objetos que se validan antes de actuar

Diferencias entre atributos y parámetros

Variables permanentes vs temporales y sus ciclos de vida

Independencia de memoria entre objetos

Cómo cada objeto mantiene su propio estado sin afectar a otros

El Operador Punto: Nuestro Control Remoto

Una vez creado el objeto con `new`, necesitamos una forma de acceder a su interior. El operador punto es la llave maestra que nos permite interactuar con los atributos y métodos del objeto. Es como tener un control remoto para manipular nuestro auto virtual.

Operaciones Básicas

- **Lectura de datos:** `System.out.println(miAuto.color);`
- **Escritura de datos:** `miAuto.color = "Rojo";`
- **Llamada a métodos:** `miAuto.acelerar();`

Este mecanismo es fundamental porque establece una comunicación directa entre nuestro código y la estructura interna del objeto. Sin el operador punto, los objetos serían cajas cerradas e inaccesibles.

Nota Importante: El operador punto solo funciona después de crear la instancia con `new`. Si intentas usarlo con una variable no inicializada, obtendrás un NullPointerException.

Valores por Defecto: La Memoria Inicial

En el video notamos algo curioso. Si creamos un Auto y pedimos imprimir su patente o su velocidad *antes* de asignarles valor, el programa no falla. ¿Magia? No, es Java siendo previsor. Cuando declaramos atributos en una clase, Java les asigna valores iniciales automáticos según su tipo de dato.

Objetos (String)

`null`

Representa la ausencia de referencia a un objeto en memoria

Números (int, double)

`0`

El valor neutro para operaciones matemáticas

Booleanos

`false`

El estado lógico inicial por defecto

Esta inicialización automática es una red de seguridad que previene comportamientos impredecibles. Sin embargo, es crucial entender que `null` puede causar problemas si intentamos operar con él sin verificar primero su existencia.

```
Auto autoNuevo = new Auto();
System.out.println(autoNuevo.velocidad); // Imprime: 0
System.out.println(autoNuevo.patente);   // Imprime: null
```

¡Cuidado! Trabajar con `null` sin verificaciones puede provocar errores en tiempo de ejecución. Siempre valida antes de usar objetos que puedan ser null.

Comportamiento Real: Modificando el Estado

Un error común de principiantes es creer que los métodos solo sirven para mostrar mensajes con `System.out.println()`. El verdadero poder de un método reside en su capacidad para **modificar los atributos internos**, es decir, cambiar el estado del objeto de forma controlada y predecible.

Ejemplo: Método acelerar()

Queremos que nuestro Auto pueda aumentar su velocidad. El método no solo debe informar, sino actuar sobre el estado interno:

```
public class Auto {
   int velocidad = 0;

   void acelerar() {
      velocidad = velocidad + 10;
   }
}
```

Cada vez que llamamos a `miAuto.acelerar()`, el objeto **recuerda** su nueva velocidad. Si lo invocamos 3 veces consecutivas, la velocidad será 30.

Estado Inicial

velocidad = 0

Primera Aceleración

velocidad = 10

Segunda Aceleración

velocidad = 20

Tercera Aceleración

velocidad = 30

> Los métodos que modifican estado son el corazón del paradigma orientado a objetos. Permiten que los objetos evolucionen y respondan a estímulos externos manteniendo su integridad interna.

Métodos Inteligentes: Programación Defensiva

Aquí unimos la Unidad 2 (estructuras condicionales If/Else) con la Unidad 3 (programación orientada a objetos). En el video planteamos una restricción lógica realista: **"No se puede acelerar si el auto está apagado"**. Esta regla introduce un concepto fundamental: el objeto es responsable de validarse a sí mismo antes de ejecutar acciones.

La programación defensiva significa que nuestros métodos no confían ciegamente en el mundo exterior. En lugar de eso, verifican condiciones previas (precondiciones) antes de modificar el estado. Esto previene estados inconsistentes y errores difíciles de rastrear.

1

Sin Validación

El método acelera sin verificar, permitiendo estados inválidos

2

Con Validación

El objeto se pregunta: "¿Estoy en condiciones de ejecutar esta acción?"

3

Estado Seguro

Solo se modifica el estado si todas las condiciones se cumplen

Código Robusto con Condicionales

```
public class Auto {
    boolean encendido = false;
    int velocidad = 0;

    void encender() {
        encendido = true;
    }

    void acelerar() {
        if (encendido == true) {
            velocidad += 10;
            System.out.println("Velocidad: " + velocidad);
        } else {
            System.out.println("Error: el auto está apagado");
        }
    }
}
```

Observa cómo el método `acelerar()` primero consulta el atributo `encendido`. Solo si el auto está en marcha, procede a incrementar la velocidad. De lo contrario, emite un mensaje de error sin alterar el estado. Este es un ejemplo perfecto de **encapsulamiento con lógica de negocio**.

Parámetros vs Atributos: Entendiendo las Variables

A veces el método necesita información externa para trabajar correctamente. Aquí es donde aparece una confusión frecuente entre dos conceptos fundamentales que, aunque relacionados, tienen ciclos de vida completamente diferentes.

Atributos

- Variables que viven **permanentemente** dentro del objeto
- Se declaran al principio de la clase
- Persisten mientras el objeto exista en memoria
- Ejemplo: `int velocidad;`
- Accesibles desde cualquier método de la clase

Parámetros

- Variables **temporales** que solo existen durante la ejecución del método
- Se declaran en los paréntesis del método
- Desaparecen al finalizar el método
- Ejemplo: `acelerar(int cuanto)`
- Solo visibles dentro del método que los recibe

Ejemplo Flexible con Parámetros

```
void acelerar(int cuanto) {
    if (encendido) {
        velocidad = velocidad + cuanto;
        // 'velocidad' (Atributo) acumula y persiste
        // 'cuanto' (Parámetro) se usa y desaparece
    }
}
```

1 Llamada al Método

`miAuto.acelerar(25);`

El parámetro 'cuanto' recibe el valor 25

2 Ejecución

Se suma 25 a velocidad

El atributo persiste con el nuevo valor

3 Finalización

El parámetro 'cuanto' desaparece

El atributo 'velocidad' permanece modificado

Regla de Oro: Si necesitas que un valor persista entre llamadas a métodos, debe ser un atributo. Si solo lo necesitas temporalmente para un cálculo, usa un parámetro.

Identidad de Objetos: La Prueba de los Dos Autos

Para cerrar este recorrido, el video demuestra uno de los principios más importantes de la programación orientada a objetos: la **independencia de memoria**. Si fabricamos dos objetos de la misma clase, lo que le suceda a uno no afecta al otro. Son entidades completamente distintas en la RAM, cada una con su propio espacio de memoria y su propio estado interno.

Este concepto es fundamental porque garantiza que los objetos sean verdaderamente autónomos. Imagina que estás gestionando una flota de vehículos: cada Auto debe poder modificar su velocidad, consumo de combustible o estado de encendido sin interferir con los demás. Esta independencia es lo que hace escalable y mantenible el código orientado a objetos.

Simulación Práctica

```
Auto ferrari = new Auto();
Auto fitito = new Auto();

ferrari.encender();
ferrari.acelerar(100);

// Pregunta de examen: ¿Cuánto vale la velocidad del fitito?
System.out.println(fitito.velocidad);
```

Ferrari	Fitito
`encendido = true`	`encendido = false`
`velocidad = 100`	`velocidad = 0`
Ha sido encendido y acelerado	Permanece en su estado inicial intacto

Respuesta: La velocidad del fitito vale `0`. Que el Ferrari acelere hasta 100 km/h no mueve al Fitito ni un centímetro. Esto confirma los principios de **Encapsulamiento** e **Identidad**: cada objeto es dueño y señor de su propio estado, con una identidad única en memoria que lo distingue de cualquier otro objeto, incluso si provienen de la misma clase.

> "La identidad de un objeto es tan única como una huella digital. Dos objetos pueden ser idénticos en apariencia (misma clase, mismos valores), pero siguen siendo entidades separadas e independientes en memoria."

Resumen: Principios Fundamentales

Hemos recorrido los conceptos esenciales que dan vida a los objetos en Java. Desde el acceso mediante el operador punto hasta la validación interna con lógica defensiva, cada elemento contribuye a crear código robusto, mantenible y profesional. Estos principios no son solo teoría académica: son las herramientas que usarás diariamente como programador.

Operador Punto

La interfaz de acceso a atributos y métodos de un objeto

Valores por Defecto

Java inicializa automáticamente: null para objetos, 0 para números, false para booleanos

Métodos que Modifican Estado

El comportamiento real no solo imprime, sino que transforma el estado interno

Programación Defensiva

Los objetos validan sus precondiciones antes de ejecutar acciones

Atributos vs Parámetros

Los atributos persisten, los parámetros son temporales

Identidad e Independencia

Cada objeto tiene su propia región de memoria y estado autónomo

Con estos fundamentos sólidos, estás preparado para crear objetos que no solo existen en memoria, sino que **viven, responden y evolucionan** de forma inteligente. La programación orientada a objetos te permite modelar el mundo real con código que es intuitivo, escalable y elegante.